

Configuring the STA Environment

This chapter describes how to set up the environment for static timing analysis. Specification of correct constraints is important in analyzing STA results. The design environment should be specified accurately so that STA analysis can identify all the timing issues in the design. Preparing for STA involves amongst others, setting up clocks, specifying IO timing characteristics, and specifying false paths and multicycle paths. It is important to understand this chapter thoroughly before proceeding with the next chapter on timing verification.

7.1 What is the STA Environment?

Most digital designs are synchronous where the data computed from the previous clock cycle is latched in the flip-flops at the active clock edge. Consider a typical synchronous design shown in Figure 7-1. It is assumed that the Design Under Analysis (DUA) interacts with other synchronous designs. This means that the DUA receives the data from a clocked flip-flop and outputs data to another clocked flip-flop external to the DUA.

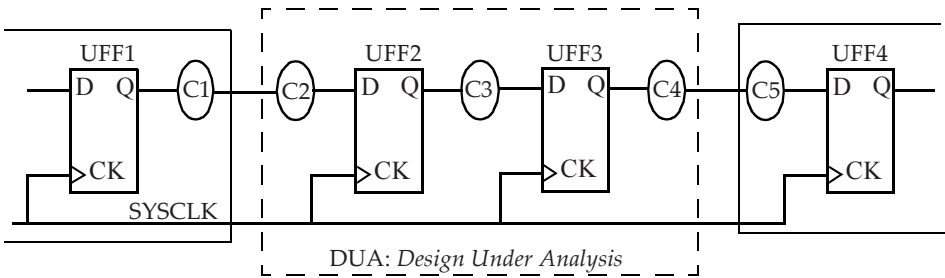


Figure 7-1 A synchronous design.

To perform STA on this design, one needs to specify the clocks to the flip-flops, and timing constraints for all paths leading into the design and for all paths exiting the design.

The example in Figure 7-1 assumes that there is only one clock and C1, C2, C3, C4, and C5 represent combinational blocks. The combinational blocks C1 and C5 are outside of the design being analyzed.

In a typical design, there can be multiple clocks with many paths from one clock domain to another. The following sections describe how the environment is specified in such scenarios.

7.2 Specifying Clocks

To define a clock, we need to provide the following information:

- i. *Clock source*: it can be a port of the design, or be a pin of a cell inside the design (typically that is part of a clock generation logic).
- ii. *Period*: the time period of the clock.
- iii. *Duty cycle*: the high duration (positive phase) and the low duration (negative phase).
- iv. *Edge times*: the times for the rising edge and the falling edge.

Figure 7-2 shows the basic definitions. By defining the clocks, all the internal timing paths (all flip-flop to flip-flop paths) are constrained; this implies that all internal paths can be analyzed with just the clock specifications. The clock specification specifies that a flip-flop to flip-flop path must take one cycle. We shall later describe how this requirement (of one cycle timing) can be relaxed.

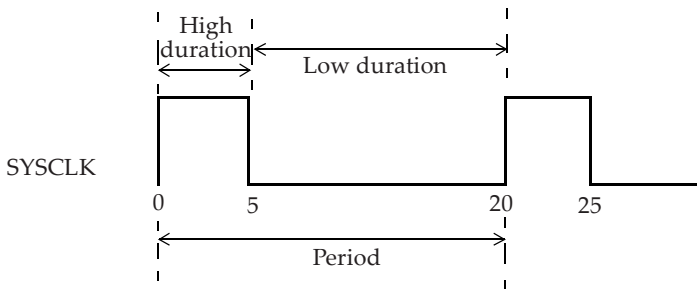


Figure 7-2 A clock definition.

Here is a basic clock specification¹.

```
create_clock \  
-name SYSCLK \  
-period 20 \  
-waveform {0 5} \  
[get_ports2 SCLK]
```

The name of the clock is *SYSCLK* and is defined at the port *SCLK*. The period of *SYSCLK* is specified as 20 units - the default time unit is nanoseconds if none has been specified. (In general, the time unit is specified as part of the technology library.) The first argument in the waveform specifies the time at which rising edge occurs and the second argument specifies the time at which the falling edge occurs.

There can be any number of edges specified in a waveform option. However all the edges must be within one period. The edge times alternate starting from the first rising edge after time zero, then a falling edge, then a rising edge, and so on. This implies that all time values in the edge list must be monotonically increasing.

```
-waveform {time_rise time_fall time_rise time_fall ...}
```

In addition, there must be an even number of edges specified. The *waveform* option specifies the waveform within one clock period, which then repeats itself.

If no *waveform* option is specified, the default is:

```
-waveform {0, period/2}
```

1. The *specification* and *constraint* are used as synonyms to each other. These are all part of the SDC specifications.

2. See appendix on SDC regarding scenarios when object access commands, such as *get_ports* and *get_clocks*, should be used.

Here is an example of a clock specification with no waveform specification (see Figure 7-3).

```
create_clock -period 5 [get_ports SCAN_CLK]
```

In this specification, since no *-name* option is specified, the name of the clock is the same as the name of the port, which is *SCAN_CLK*.

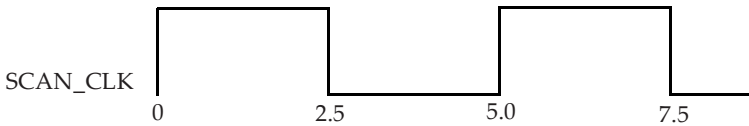


Figure 7-3 *Clock specification example.*

Here is another example of a clock specification in which the edges of the waveform are in the middle of a period (see Figure 7-4).

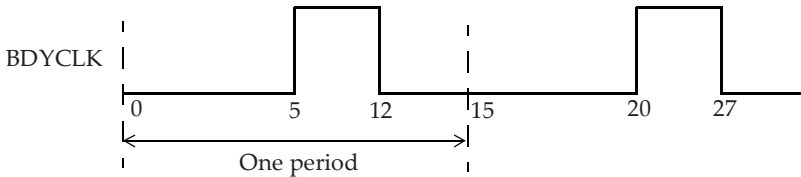


Figure 7-4 *Clock specification with arbitrary edges.*

```
create_clock -name BDYCLK -period 15 \  
-waveform {5 12} [get_ports GBLCLK]
```

The name of the clock is *BDYCLK* and it is defined at the port *GBLCLK*. In practice, it is a good idea to keep the clock name the same as the port name.

Here are some more clock specifications.

```
# See Figure 7-5(a):
create_clock -period 10 -waveform {5 10} [get_ports FCLK]
# Creates a clock with the rising edge at 5ns and the
# falling edge at 10ns.

# See Figure 7-5(b):
create_clock -period 125 \
  -waveform {100 150} [get_ports ARMCLK]
# Since the first edge has to be rising edge,
# the edge at 100ns is specified first and then the
# falling edge at 150ns is specified. The falling edge
# at 25ns is automatically inferred.
```

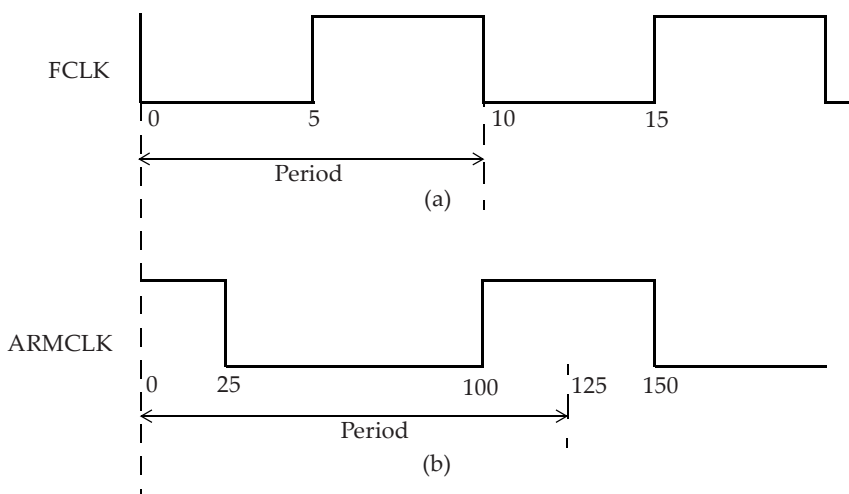


Figure 7-5 *Example clock waveforms.*

```
# See Figure 7-6(a):
create_clock -period 1.0 -waveform {0.5 1.375} MAIN_CLK
# The first rising edge and the next falling edge
```

is specified. Falling edge at 0.375ns is inferred
automatically.

See Figure 7-6(b):

```
create_clock -period 1.2 -waveform {0.3 0.4 0.8 1.0} JTAG_CLK  
# Indicates a rising edge at 300ps, a falling edge at 400ps,  
# a rising edge at 800ps and a falling edge at 1ns, and this  
# pattern is repeated every 1.2ns.
```

```
create_clock -period 1.27 \  
-waveform {0 0.635} [get_ports clk_core]
```

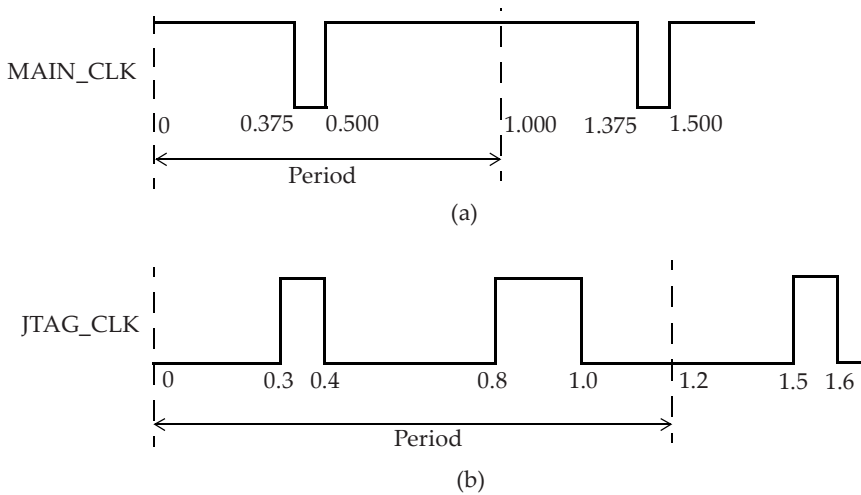


Figure 7-6 *Example with general clock waveforms.*

```
create_clock -name TEST_CLK -period 17 \  
-waveform {0 8.5} -add [get_ports {ip_io_clk[0]}]  
# The -add option allows more than one clock  
# specification to be defined at a port.
```

In addition to the above attributes, one can optionally specify the transition time (slew) at the source of the clock. In some cases, such as the output of some PLL¹ models or an input port, the tool cannot compute the transition time automatically. In such cases, it is useful to explicitly specify the transition time at the source of the clock. This is specified using the **set_clock_transition** specification.

```
set_clock_transition -rise 0.1 [get_clocks CLK_CONFIG]  
set_clock_transition -fall 0.12 [get_clocks CLK_CONFIG]
```

This specification applies only for ideal clocks and is disregarded once the clock trees are built, at which point, actual transition times at the clock pins are used. If a clock is defined on an input port, use the *set_input_transition* specification (see Section 7.7) to specify the slew on the clock.

7.2.1 Clock Uncertainty

The timing uncertainty of a clock period can be specified using the **set_clock_uncertainty** specification. The uncertainty can be used to model various factors that can reduce the effective clock period. These factors can be the clock jitter and any other pessimism that one may want to include for timing analysis.

```
set_clock_uncertainty -setup 0.2 [get_clocks CLK_CONFIG]  
set_clock_uncertainty -hold 0.05 [get_clocks CLK_CONFIG]
```

Note that the clock uncertainty for setup effectively reduces the available clock period by the specified amount as illustrated in Figure 7-7. For hold checks, the clock uncertainty for hold is used as an additional timing margin that needs to be satisfied.

1. Phase-locked loop: commonly used in an ASIC to generate high-frequency clocks.

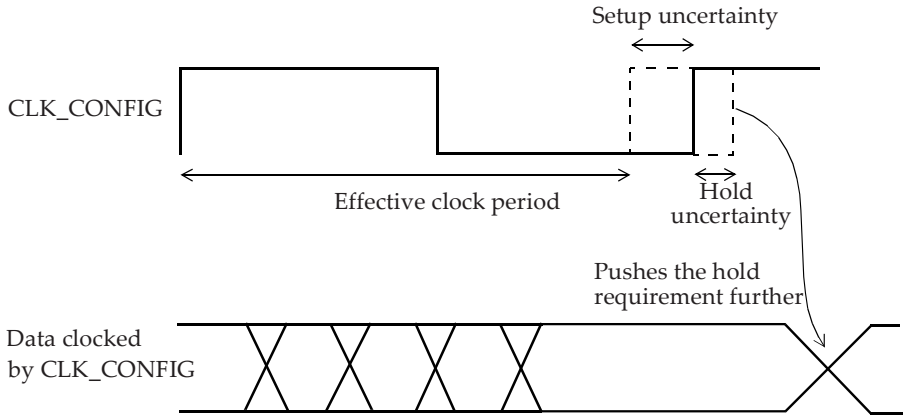


Figure 7-7 *Specifying clock uncertainty.*

The following commands specify the uncertainty to be used on paths crossing the specified clock boundaries, called **inter-clock uncertainty**.

```
set_clock_uncertainty -from VIRTUAL_SYS_CLK -to SYS_CLK \
    -hold 0.05
set_clock_uncertainty -from VIRTUAL_SYS_CLK -to SYS_CLK \
    -setup 0.3
set_clock_uncertainty -from SYS_CLK -to CFG_CLK -hold 0.05
set_clock_uncertainty -from SYS_CLK -to CFG_CLK -setup 0.1
```

Figure 7-8 shows a path between two different clock domains, *SYS_CLK* and *CFG_CLK*. Based on the above inter-clock uncertainty specifications, 100ps is used as an uncertainty for setup checks and 50ps is used as an uncertainty for hold checks.

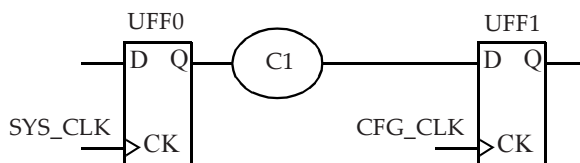


Figure 7-8 *Inter-clock paths.*

7.2.2 Clock Latency

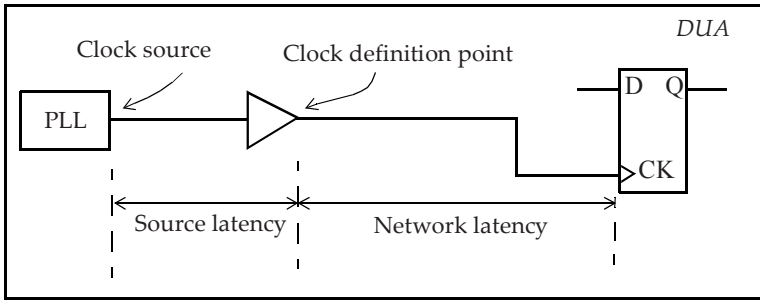
Latency of a clock can be specified using the `set_clock_latency` command.

```
# Rise clock latency on MAIN_CLK is 1.8ns:
set_clock_latency 1.8 -rise [get_clocks MAIN_CLK]
# Fall clock latency on all clocks is 2.1ns:
set_clock_latency 2.1 -fall [all_clocks]
# The -rise, -fall refer to the edge at the clock pin of a
# flip-flop.
```

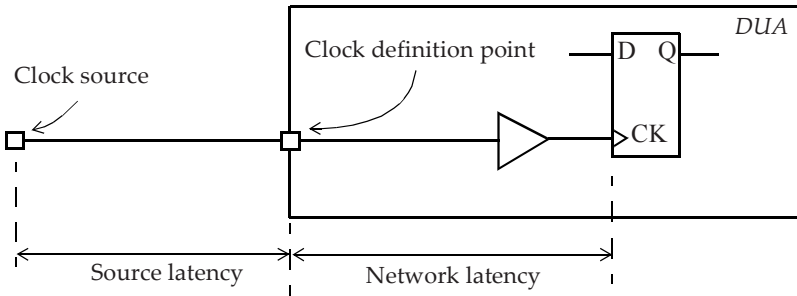
There are two types of clock latencies: **network latency** and **source latency**. Network latency is the delay from the clock definition point (*create_clock*) to the clock pin of a flip-flop. Source latency, also called **insertion delay**, is the delay from the clock source to the clock definition point. Source latency could represent either on-chip or off-chip latency. Figure 7-9 shows both the scenarios. The total clock latency at the clock pin of a flip-flop is the sum of the source and network latencies.

Here are some example commands that specify source and network latencies.

```
# Specify a network latency (no -source option) of 0.8ns for
# rise, fall, max and min:
set_clock_latency 0.8 [get_clocks CLK_CONFIG]
# Specify a source latency:
set_clock_latency 1.9 -source [get_clocks SYS_CLK]
# Specify a min source latency:
```



(a) On-chip clock source.



(b) Off-chip clock source.

Figure 7-9 Clock latencies.

```
set_clock_latency 0.851 -source -min [get_clocks CFG_CLK]
# Specify a max source latency:
set_clock_latency 1.322 -source -max [get_clocks CFG_CLK]
```

One important distinction to observe between source and network latency is that once a clock tree is built for a design, the network latency can be ignored (assuming *set_propagated_clock* command is specified). However, the source latency remains even after the clock tree is built. The network latency is an estimate of the delay of the clock tree prior to clock tree synthesis. After clock tree synthesis, the total clock latency from clock source to a

clock pin of a flip-flop is the source latency plus the actual delay of the clock tree from the clock definition point to the flip-flop.

Generated clocks are described in the next section and virtual clocks are described in Section 7.9.

7.3 Generated Clocks

A **generated clock** is a clock derived from a master clock. A master clock is a clock defined using the *create_clock* specification.

When a new clock is generated in a design that is based on a master clock, the new clock can be defined as a generated clock. For example, if there is a divide-by-3 circuitry for a clock, one would define a generated clock definition at the output of this circuitry. This definition is needed as STA does not know that the clock period has changed at the output of the divide-by logic, and more importantly what the new clock period is. Figure 7-10 shows an example of a generated clock which is a divide-by-2 of the master clock, *CLKP*.

```
create_clock -name CLKP 10 [get_pins UPLL0/CLKOUT]
# Create a master clock with name CLKP of period 10ns
# with 50% duty cycle at the CLKOUT pin of the PLL.
create_generated_clock -name CLKPDIV2 -source UPLL0/CLKOUT \
-divide_by 2 [get_pins UFF0/Q]
# Creates a generated clock with name CLKPDIV2 at the Q
# pin of flip-flop UFF0. The master clock is at the CLKOUT
# pin of PLL. And the period of the generated clock is double
# that of the clock CLKP, that is, 20ns.
```

Can a new clock, that is, a master clock, be defined at the output of the flip-flop instead of a generated clock? The answer is yes, that it is indeed possible. However, there are some disadvantages. Defining a master clock instead of a generated clock creates a new clock domain. This is not a problem in general except that there are more clock domains to deal with

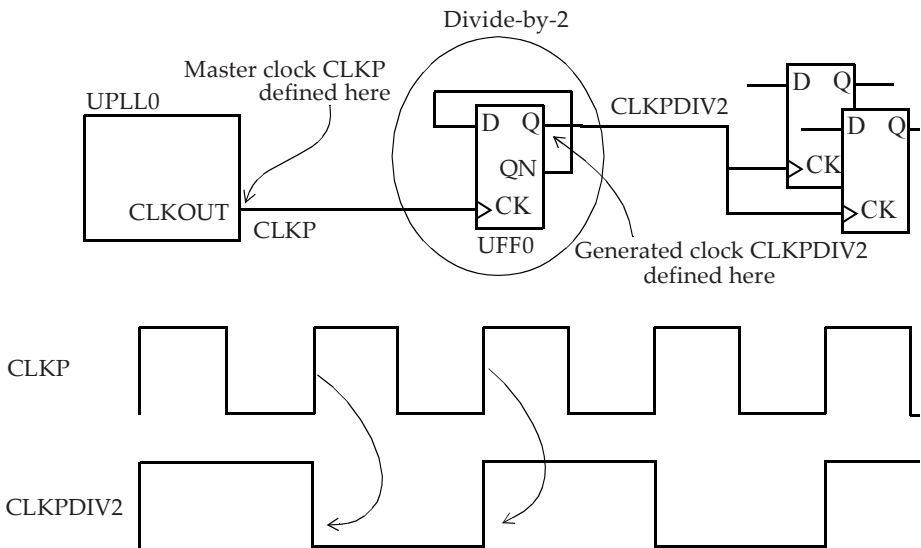


Figure 7-10 *Generated clock at output of divider.*

in setting up the constraints for STA. Defining the new clock as a generated clock does not create a new clock domain, and the generated clock is considered to be in phase with its master clock. The generated clock does not require additional constraints to be developed. Thus, one must attempt to define a new internally generated clock as a *generated clock* instead of deciding to declare it as another master clock.

Another important difference between a master clock and a generated clock is the notion of clock origin. In a master clock, the origin of the clock is at the point of definition of the master clock. In a generated clock, the clock origin is that of the master clock and not that of the generated clock. This implies that in a clock path report, the start point of a clock path is always the master clock definition point. This is a big advantage of a generated clock over defining a new master clock as the source latency is not automatically included for the case of a new master clock.

Figure 7-11 shows an example of a multiplexer with clocks on both its inputs. In this case, it is not necessary to define a clock on the output of the multiplexer. If the select signal is set to a constant, the output of the multiplexer automatically gets the correct clock propagated. If the select pin of the multiplexer is unconstrained, both the clocks propagate through the multiplexer for the purposes of the STA. In such cases, the STA may report paths between *TCLK* and *TCLKDIV5*. Note that such paths are not possible as the select line can select only one of the multiplexer inputs. In such a case, one may need to set a false path or specify an exclusive clock relationship between these two clocks to avoid incorrect paths being reported. This of course assumes that there are no paths between *TCLK* and *TCLKDIV5* elsewhere in the design.

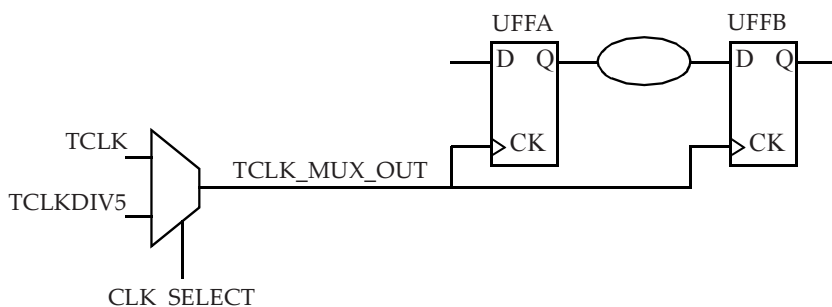


Figure 7-11 *Multiplexer selecting between two clocks.*

What happens if the multiplexer select signal is not static and can change during device operation? In such cases, clock gating checks are inferred for the multiplexer inputs. Clock gating checks are explained in Chapter 10; these checks ensure that the clocks at the multiplexer inputs switch safely with respect to the multiplexer select signal.

Figure 7-12 shows an example where the clock *SYS_CLK* is gated by the output of a flip-flop. Since the output of the flip-flop may not be a constant, one way to handle this situation is to define a generated clock at the output of the *and* cell which is identical to the input clock.

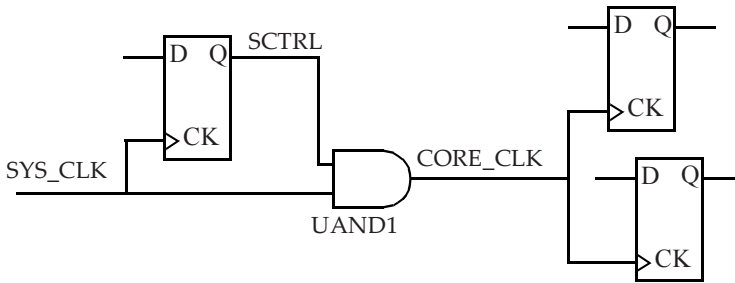


Figure 7-12 *Clock gated by a flip-flop.*

```
create_clock 0.1 [get_ports SYS_CLK]
# Create a master clock of period 100ps with 50%
# duty cycle.
create_generated_clock -name CORE_CLK -divide_by 1 \
-source SYS_CLK [get_pins UAND1/Z]
# Create a generated clock called CORE_CLK at the
# output of the and cell and the clock waveform is
# same as that of the master clock.
```

The next example is of a generated clock that has a frequency higher than that of the source clock. Figure 7-13 shows the waveforms.

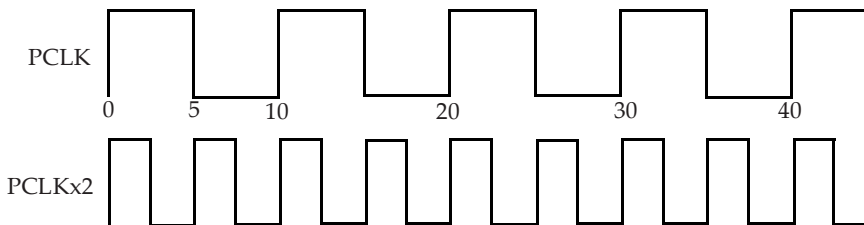


Figure 7-13 *Master clock and multiply-by-2 generated clock.*

```
create_clock -period 10 -waveform {0 5} [get_ports PCLK]
# Create a master clock with name PCLK of period 10ns
# with rise edge at 0ns and fall edge at 5ns.
create_generated_clock -name PCLKx2 \
  -source [get_ports PCLK] \
  -multiply_by 2 [get_pins UCLKMULTREG/Q]
# Creates a generated clock called PCLKx2 from the
# master clock PCLK and the frequency is double that of
# the master clock. The generated clock is defined at the
# output of the flip-flop UCLKMULTREG.
```

Note that the *-multiply_by* and the *-divide_by* options refer to the frequency of the clock, even though a clock period is specified in a master clock definition.

Example of Master Clock at Clock Gating Cell Output

Consider the clock gating example shown in Figure 7-14. Two clocks are fed to an *and* cell. The question is what is at the output of the *and* cell. If the input to the *and* cell are both clocks, then it is safe to define a new main clock at the output of the *and* cell, since it is highly unlikely that the output of the cell has any phase relationship with either of the input clocks.

```
create_clock -name SYS_CLK -period 4 -waveform {0 2} \
  [get_pins UFFSYS/Q]
create_clock -name CORE_CLK -period 12 -waveform {0 4} \
  [get_pins UFFCORE/Q]
create_clock -name MAIN_CLK -period 12 -waveform {0 2} \
  [get_pins UAND2/Z]
# Create a master clock instead of a generated clock
# at the output of the and cell.
```

One drawback of creating clocks at the internal pins is that it impacts the path delay computation and forces the designer to manually compute the source latencies.

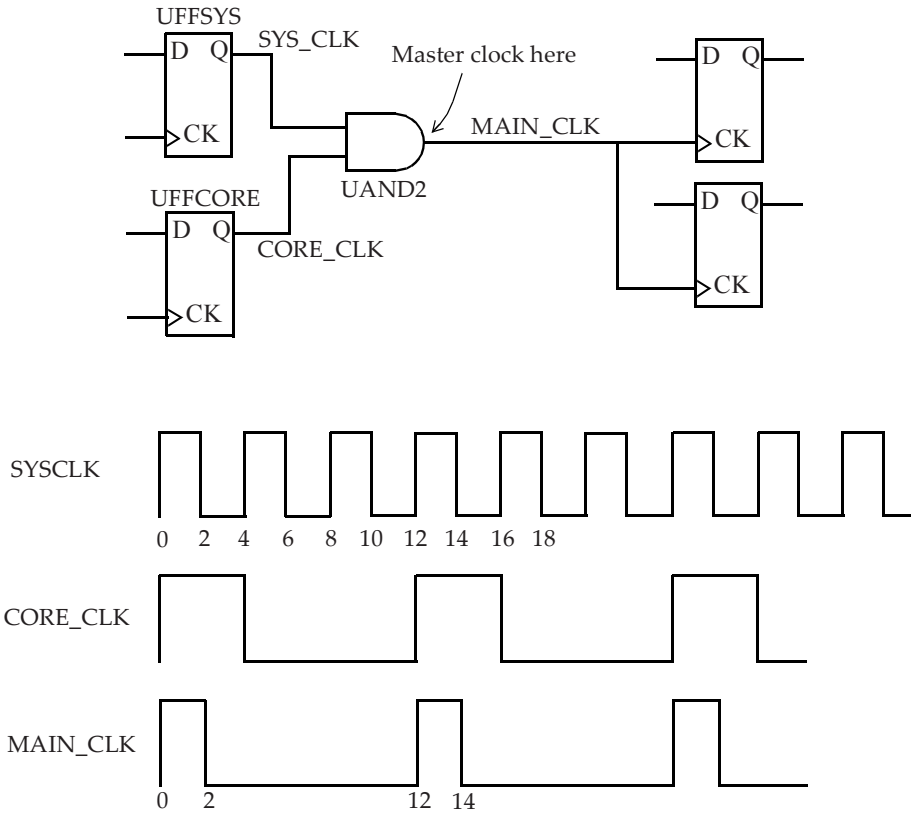


Figure 7-14 Master clock at output of logic gate.

Generated Clock using Edge and Edge_shift Options

Figure 7-15 shows an example of generated clocks. A divide-by-2 clock in addition to two out-of-phase clocks are generated. The waveforms for the clocks are also shown in the figure.

The clock definitions for this example are given below. The generated clock definition illustrates the **-edges** option, which is another way to define a generated clock. This option takes a list of edges {*rise*, *fall*, *rise*} of the source

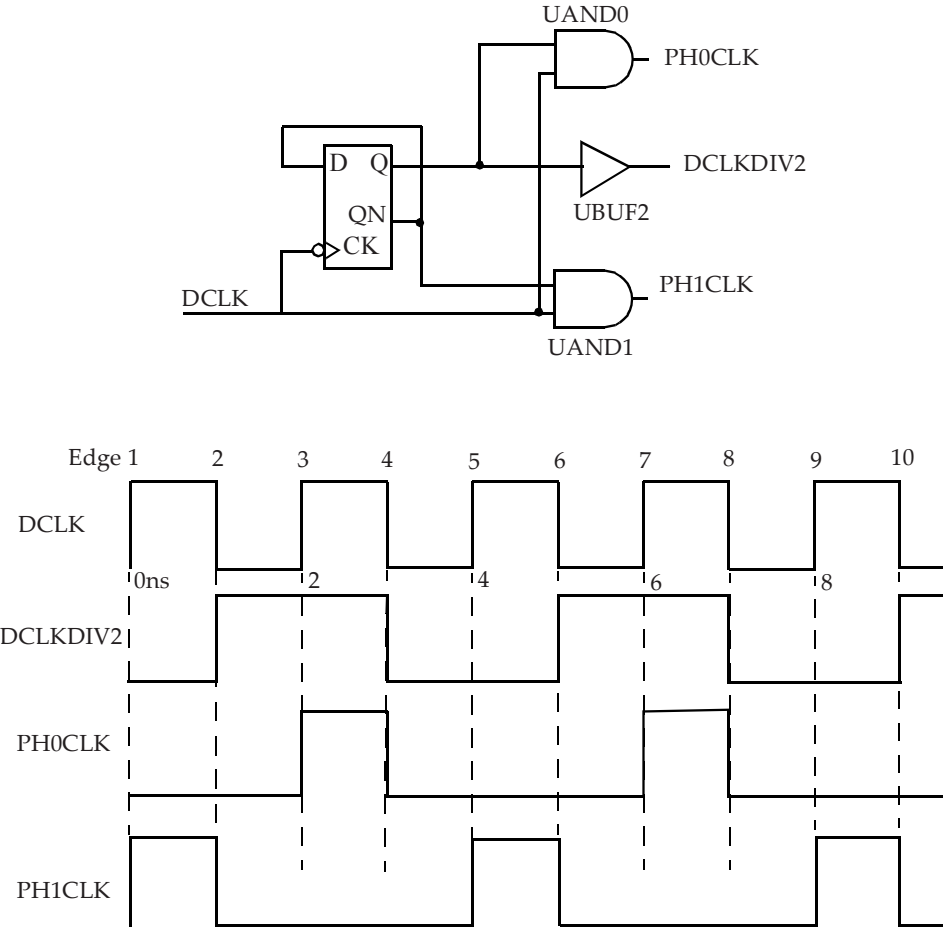


Figure 7-15 *Clock generation.*

master clock to form the new generated clock. The first rise edge of the master clock is the first edge, the first fall edge is edge 2, the next rise edge is edge 3, and so on.

```

create_clock 2 [get_ports DCLK]
# Name of clock is DCLK, has period of 2ns with a
# rise edge at 0ns and a fall edge at 1ns.
create_generated_clock -name DCLKDIV2 -edges {2 4 6} \
-source DCLK [get_pins UBUF2/Z]
# The generated clock with name DCLKDIV2 is defined at
# the output of the buffer. Its waveform is formed by
# having a rise edge at edge 2 of the source clock,
# fall edge at edge 4 of the source clock and the next
# rise edge at edge 6 of the source clock.
create_generated_clock -name PH0CLK -edges {3 4 7} \
-source DCLK [get_pins UAND0/Z]
# The generated clock PH0CLK is formed using
# the 3, 4, 7 edges of the source clock.
create_generated_clock -name PH1CLK -edges {1 2 5} \
-source DCLK [get_pins UAND1/Z]
# The generated clock with name PH1CLK is defined at
# the output of the and cell and is formed with
# edges 1, 2 and 5 of the source clock.

```

What if the first edge of the generated clock is a falling edge? Consider the generated clock *G3CLK* shown in Figure 7-16. Such a generated clock can be defined by specifying the edges 5, 7 and 10, as shown in the following clock specification. The falling edge at 1ns is inferred automatically.

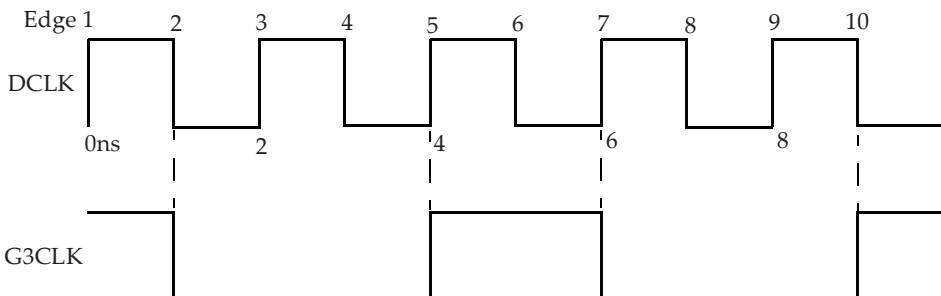


Figure 7-16 *Generated clock with a falling edge as first edge.*

```
create_generated_clock -name G3CLK -edges {5 7 10} \  
-source DCLK [get_pins UANDO/Z]
```

The **-edge_shift** option can be used in conjunction with the **-edges** option to specify any shift of the corresponding edges to form the new generated waveform. It specifies the amount of shift (in time units) for each edge in the edge list. Here is an example that uses this option.

```
create_clock -period 10 -waveform {0 5} [get_ports MIICLK]  
create_generated_clock -name MIICLKDIV2 -source MIICLK \  
-edges {1 3 5} [get_pins UMIICLKREG/Q]  
# Create a divide-by-2 clock.  
create_generated_clock -name MIIDIV2 -source MIICLK \  
-edges {1 1 5} -edge_shift {0 5 0} [get_pins UMIIDIV/Q]  
# Creates a divide-by-2 clock with a duty cycle different  
# from the source clock's value of 50%.
```

The list of edges in the edge list must be in non-decreasing order, though the same edge can be used for two entries to indicate a clock pulse independent of the source clocks' duty cycle. The **-edge_shift** option in the above example specifies that the first edge is obtained by shifting (edge 1 of source clock) by 0ns, the second edge is obtained by shifting (edge 1 of source clock) by 5ns and the third edge is obtained by shifting (edge 5 of source clock) by 0ns. Figure 7-17 shows the waveforms.

Generated Clock using Invert Option

Here is another example of a generated clock; this one uses the **-invert** option.

```
create_clock -period 10 [get_ports CLK]  
create_generated_clock -name NCLKDIV2 -divide_by 2 -invert \  
-source CLK [get_pins UINVQ/Z]
```

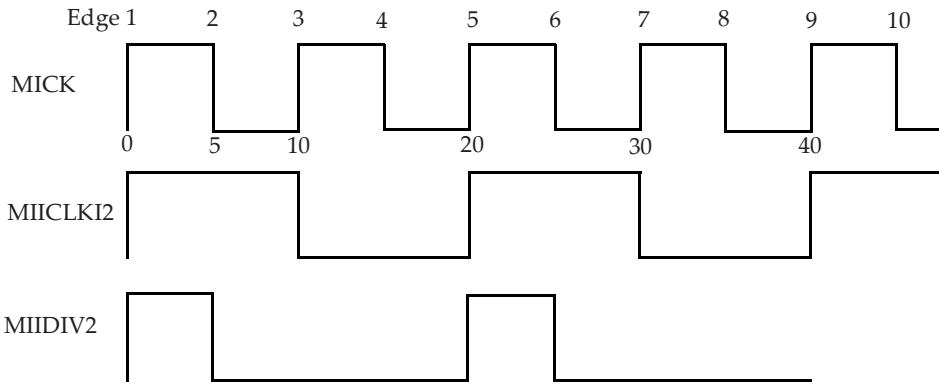


Figure 7-17 *Generated clocks using -edge_shift option.*

The *-invert* option applies the inversion to the generated clock after all other generated clock options are applied. Figure 7-18 shows a schematic that generates such an inverted clock.

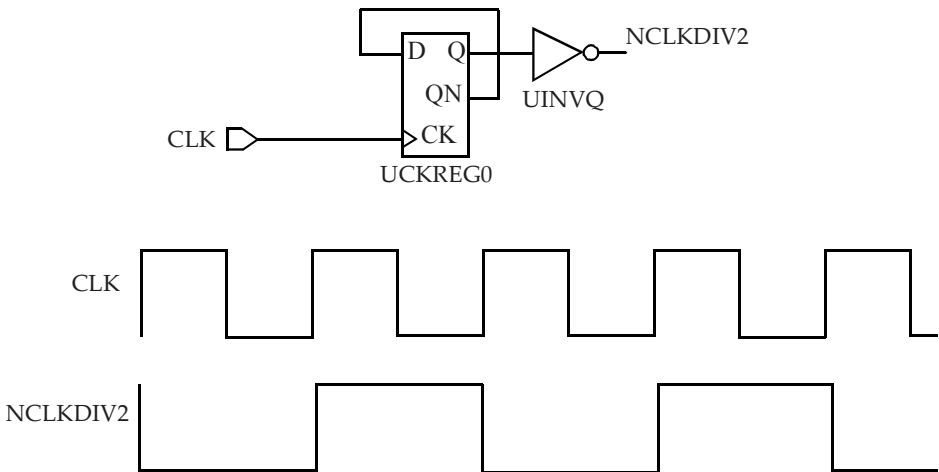


Figure 7-18 *Inverting a clock.*

Clock Latency for Generated Clocks

Clock latencies can be specified for generated clocks as well. A source latency specified on a generated clock specifies the latency from the definition of the master clock to the definition of the generated clock. The total clock latency to a clock pin of a flop-flop being driven by a generated clock is thus the sum of the source latency of the master clock, the source latency of the generated clock and the network latency of the generated clock. This is shown in Figure 7-19.

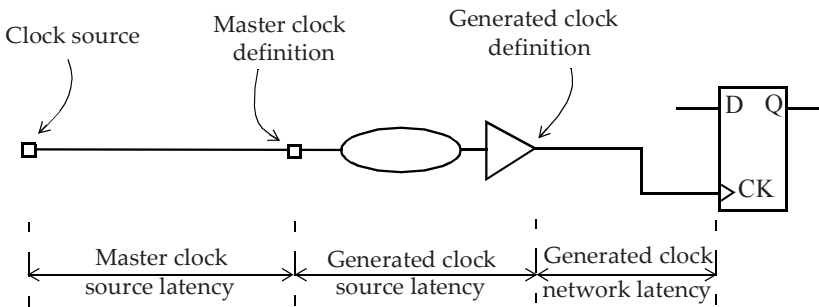


Figure 7-19 *Latency on generated clock.*

A generated clock can have another generated clock as its source, that is, one can have generated clocks of generated clocks, and so on. However, a generated clock can have only one master clock. More examples of generated clocks are described in later chapters.

Typical Clock Generation Scenario

Figure 7-20 shows a scenario of how a clock distribution may appear in a typical ASIC. The oscillator is external to the chip and produces a low frequency (10-50 MHz typical) clock which is used as a reference clock by the on-chip PLL to generate a high-frequency low-jitter clock (200-800 MHz typical). This PLL clock is then fed to a clock divider logic that generates the required clocks for the ASIC.

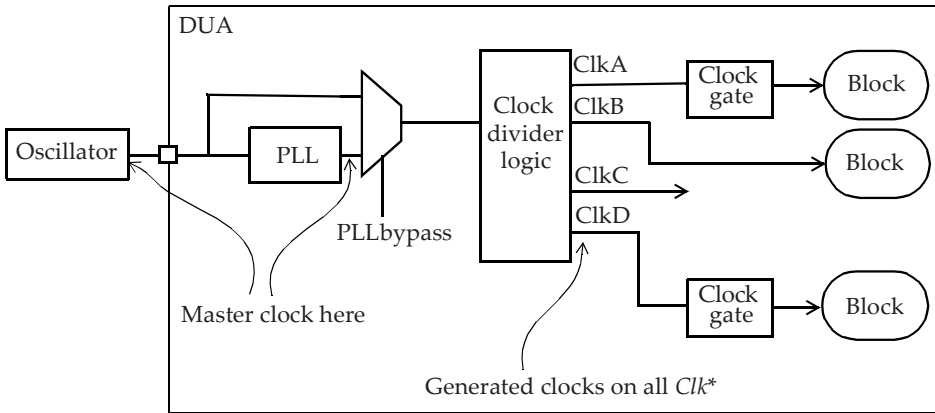


Figure 7-20 *Clock distribution in a typical ASIC.*

On some of the branches of the clock distribution, there may be clock gates that are used to turn off the clock to an inactive portion of the design to save power when necessary. The PLL can also have a multiplexer at its output so that the PLL can be bypassed if necessary.

A master clock is defined for the reference clock at the input pin of the chip where it enters the design, and a second master clock is defined at the output of the PLL. The PLL output clock has no phase relationship with the reference clock. Therefore, the output clock should *not* be a generated clock of the reference clock. Most likely, all clocks generated by the clock divider logic are specified as generated clocks of the master clock at the PLL output.

7.4 Constraining Input Paths

This section describes the constraints for the input paths. The important point to note here is that STA cannot check any timing on a path that is not constrained. Thus, all paths should be constrained to enable their analysis. Examples where one may not care about some logic and can leave such in-

puts unconstrained are described in later chapters. For example, one may not care about timing through inputs that are strictly control signals, and may determine that there is no need to specify the checks described in this section. However, this section assumes that we want to constrain the input paths.

Figure 7-21 shows an input path of the design under analysis (DUA). Flip-flop *UFF0* is external to the design and provides data to the flip-flop *UFF1* which is internal to the design. The data is connected through the input port *INP1*.

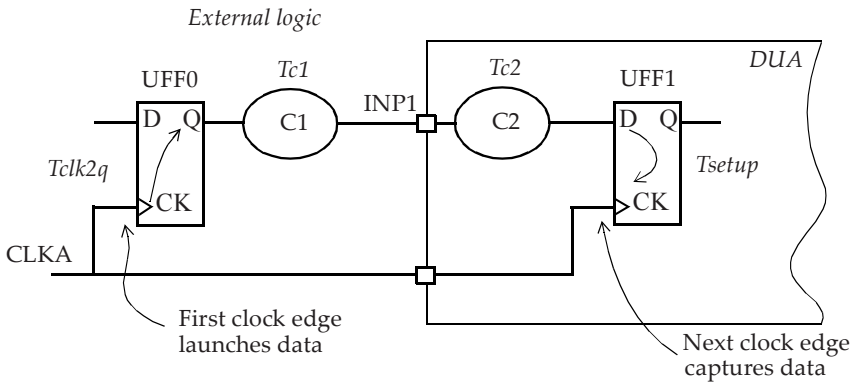


Figure 7-21 *Input port timing path.*

The clock definition for *CLKA* specifies the clock period, which is the total amount of time available between the two flip-flops *UFF0* and *UFF1*. The time taken by the external logic is *Tclk2q*, the CK to Q delay of the launch flip-flop *UFF0*, plus *Tc1*, the delay through the external combinational logic. Thus, the delay specification on an input pin *INP1* defines an external delay of *Tclk2q* plus *Tc1*. This delay is specified with respect to a clock, *CLKA* in this example.

Here is the input delay constraint.

```
set Tclk2q 0.9
set Tc1 0.6
set_input_delay -clock CLKA -max [expr Tclk2q + Tc1] \
[get_ports INP1]
```

The constraint specifies that the external delay on input *INP1* is 1.5ns and this is with respect to the clock *CLKA*. Assuming the clock period for *CLKA* is 2ns, then the logic for *INP1* pin has only 500ps (= 2ns - 1.5ns) available for propagating internally in the design. This input delay specification maps into the input constraint that *Tc2* plus *Tsetup* of *UFF1* must be less than 500ps for the flip-flop *UFF1* to reliably capture the data launched by flip-flop *UFF0*. Note that the external delay above is specified as a max quantity.

Let us consider the case when we want to consider both max and min delays, as shown in Figure 7-22. Here are the constraints for this example.

```
create_clock -period 15 -waveform {5 12} [get_ports CLKP]
set_input_delay -clock CLKP -max 6.7 [get_ports INPA]
set_input_delay -clock CLKP -min 3.0 [get_ports INPA]
```

The max and min delays for *INPA* are derived from the *CLKP* to *INPA* delays. The max and min delays refer to the longest and shortest path delays respectively. These may also normally correspond to the worst-case slow (max timing corner) and the best-case fast (min timing corner). Thus, the max delay corresponds to the longest path delay at the max corner and the min delay corresponds to the shortest path delay at the min corner. In our example, 1.1ns and 0.8ns are the max and the min delay values for the *Tck2q*. The combinational path delay *Tc1* has a max delay of 5.6ns and a min delay of 2.2ns. The waveform on *INPA* shows the window in which the data arrives at the design input and when it is expected to be stable. The max delay from *CLKP* to *INPA* is 6.7ns (= 1.1ns + 5.6ns). The min delay is 3ns (= 0.8ns + 2.2ns). These delays are specified with respect to the active edge of the clock. Given the external input delays, the available setup time

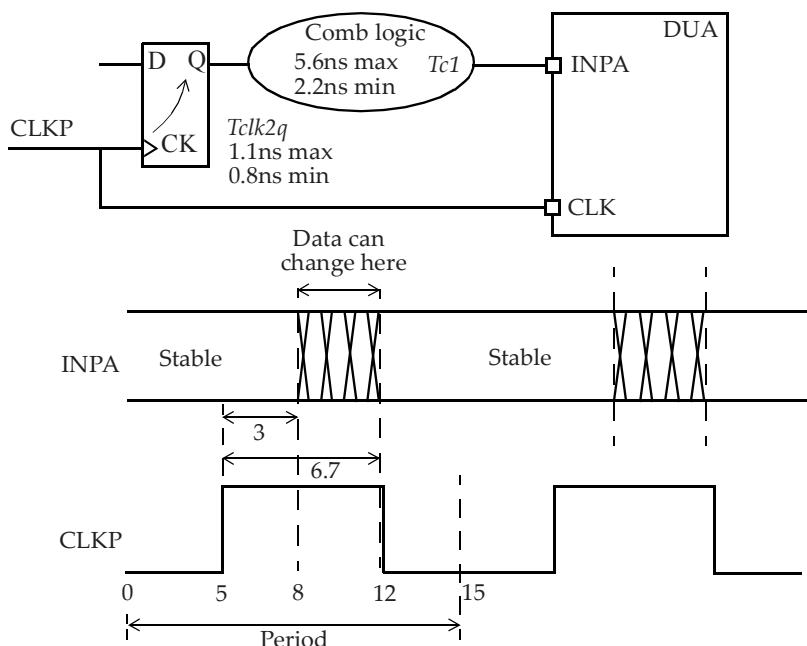


Figure 7-22 *Max and min delays on input port.*

internal to the design is the min of 8.3ns (= 15ns - 6.7ns) at the slow corner and 12ns (= 15ns - 3.0ns) at the fast corner. Thus, 8.3ns is the available time to reliably capture the data internal to the DUA.

Here are some more examples of input constraints.

```
set_input_delay -clock clk_core 0.5 [get_ports bist_model]
set_input_delay -clock clk_core 0.5 [get_ports sad_state]
```

Since the max or min options are not specified, the value of 500ps applies to both the max and min delays. This external input delay is specified with respect to the rising edge of clock *clk_core* (the **-clock_fall** option has to be

used if the input delay is specified with respect to the falling edge of the clock).

7.5 Constraining Output Paths

This section describes the constraints for the output paths with the help of three illustrative examples below.

Example A

Figure 7-23 shows an example of a path through an output port of the design under analysis. T_{c1} and T_{c2} are the delays through the combinational logic.

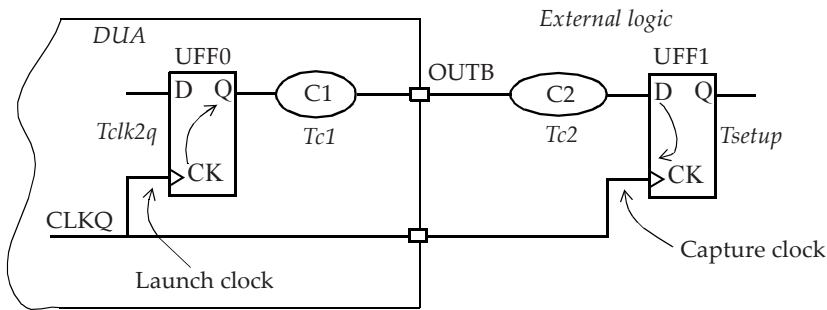


Figure 7-23 Output port timing path for example A.

The period for the clock $CLKQ$ defines the total available time between the flip-flops $UFF0$ and $UFF1$. The external logic has a total delay of T_{c2} plus T_{setup} . This total delay, $T_{c2} + T_{setup}$, has to be specified as part of the output delay specification. Note that the output delay is specified relative to the capture clock. Data must arrive at the external flip-flop $UFF1$ in time to meet its setup requirement.

```
set Tc2      3.9
set Tsetup   1.1
set_output_delay -clock CLKQ -max [expr Tc2 + Tsetup] \
  [get_ports OUTB]
```

This specifies that the max external delay relative to the clock edge is *Tc2* plus *Tsetup*; and should correspond to the delay of 5ns. A min delay can be similarly specified.

Example B

Figure 7-24 shows an example with both min and max delays. The max path delay is 7.4ns (= max *Tc2* plus *Tsetup* = 7 + 0.4). The min path delay is -0.2ns (= min *Tc2* minus *Thold* = 0 - 0.2). Therefore the output specifications are:

```
create_clock -period 20 -waveform {0 15} [get_ports CLKQ]
set_output_delay -clock CLKQ -min -0.2 [get_ports OUTC]
set_output_delay -clock CLKQ -max 7.4 [get_ports OUTC]
```

The waveforms in Figure 7-24 show when *OUTC* has to be stable so that it is reliably captured by the external flip-flop. This depicts that the data must be ready at the output port before the required stable region starts and must remain stable until the end of the stable region. This maps into a requirement on the timing of the logic to the output port *OUTC* inside the DUA.

Example C

Here is another example that shows input and output specifications. This block has two inputs, *DATAIN* and *MCLK*, and one output *DATAOUT*. Figure 7-25 shows the intended waveforms.

```
create_clock -period 100 -waveform {5 55} [get_ports MCLK]
set_input_delay 25 -max -clock MCLK [get_ports DATAIN]
set_input_delay 5 -min -clock MCLK [get_ports DATAIN]
```



7.6 Timing Path Groups

In STA, the paths are timed based upon valid startpoints and valid endpoints. Valid startpoints are: input ports and clock pins of synchronous de-

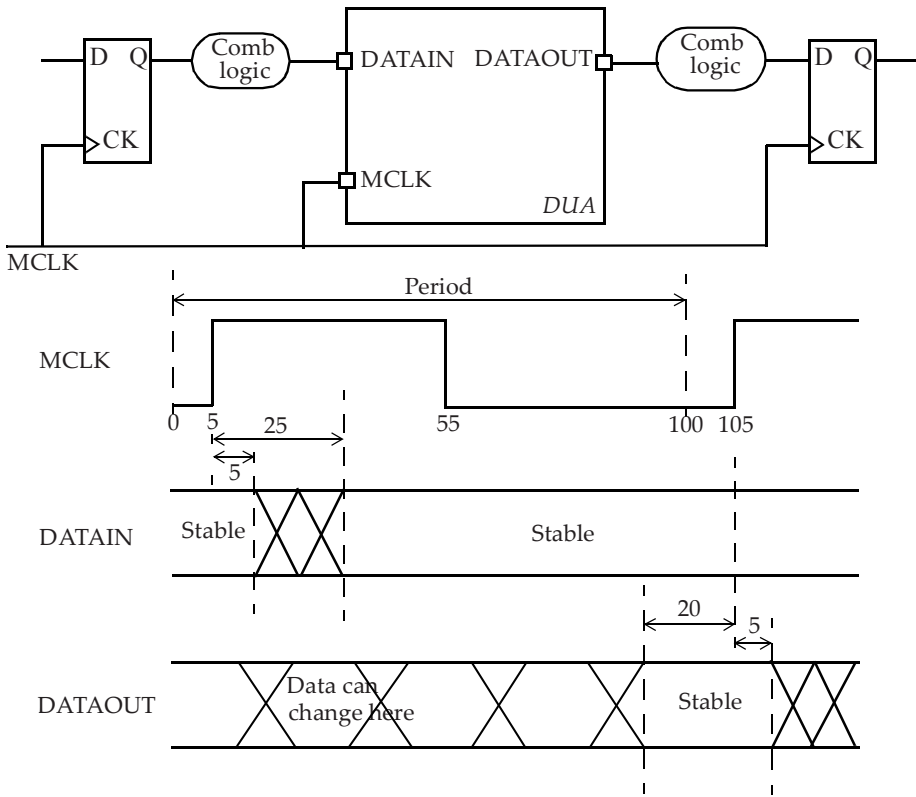


Figure 7-25 Example C with input and output specifications.

vices, such as flip-flops and memories. Valid endpoints are output ports and data input pins of synchronous devices. Thus, a valid timing path can be:

- i. from an input port to an output port,
- ii. from an input port to an input of a flip-flop or a memory,
- iii. from the clock pin of a flip-flop or a memory to an input of flip-flop or a memory,
- iv. from the clock pin of a flip-flop to an output port,

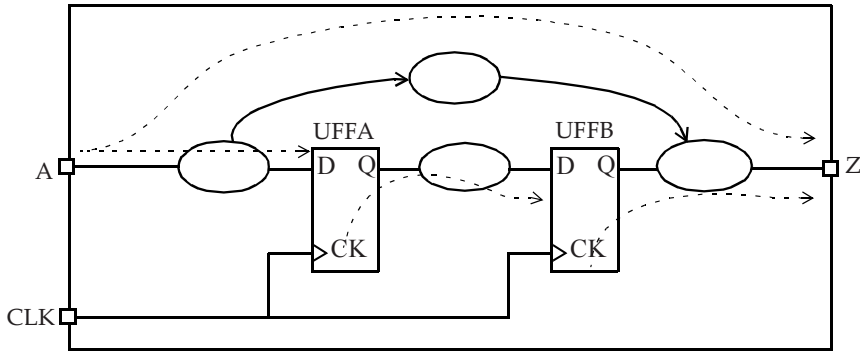


Figure 7-26 *Timing paths.*

v. from the clock pin of a memory to an output port, and so on.

The valid paths in Figure 7-26 are:

- input port A to UFFA/D,
- input port A to output port Z,
- UFFA/CLK to UFFB/D, and
- UFFB/CLK to output port Z.

Timing paths are sorted into **path groups** by the clock associated with the endpoint of the path. Thus, each clock has a set of paths associated with it. There is also a **default path group** that includes all non-clocked (asynchronous) paths.

In the example of Figure 7-27, the path groups are:

- *CLKA* group: Input port A to UFFA/D.
- *CLKB* group: UFFA/CK to UFFB/D.
- *DEFAULT* group: Input port A to output port Z, UFFB/CK to output port Z.

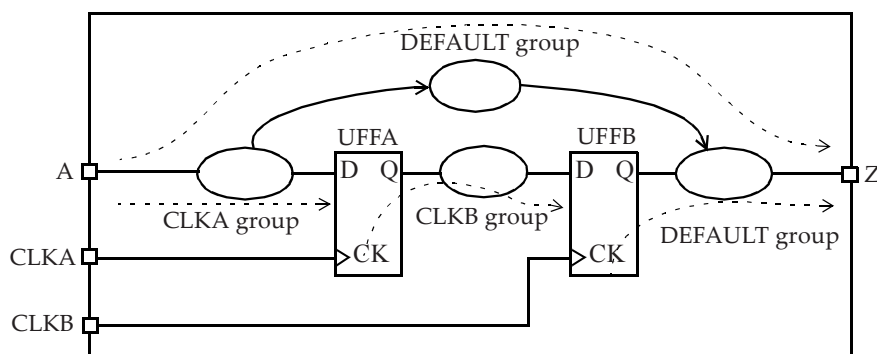


Figure 7-27 Path groups.

The static timing analysis and reporting are typically performed on each path group separately.

7.7 Modeling of External Attributes

While *create_clock*, *set_input_delay* and *set_output_delay* are enough to constrain all paths in a design for performing timing analysis, these are not enough to obtain accurate timing for the IO pins of the block. The following attributes are also required to accurately model the environment of a design. For inputs, one needs to specify the slew at the input. This information can be provided using:

- *set_drive*¹
- *set_driving_cell*
- *set_input_transition*

1. This command is obsolete and not recommended.

For outputs, one needs to specify the capacitive load seen by the output pin. This is specified by using the following specification:

- `set_load`

7.7.1 Modeling Drive Strengths

The `set_drive` and `set_driving_cell` specifications are used to model the drive strength of the external source that drives an input port of the block. In absence of these specifications, by default, all inputs are assumed to have an infinite drive strength. The default condition implies that the transition time at the input pins is 0.

The `set_drive` explicitly specifies a value for the drive resistance at the input pin of the DUA. The smaller the drive value, the higher the drive strength. A resistance value of 0 implies an infinite drive strength.

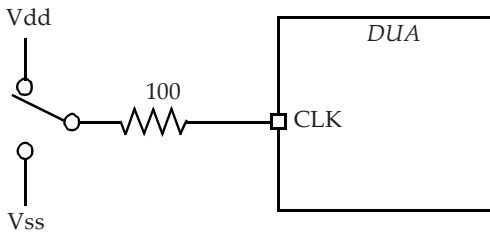


Figure 7-28 Representation for `set_drive` specification example.

```
set_drive 100 UCLK
```

```
# Specifies a drive resistance of 100 on input UCLK.
```

```
# Rise drive is different from fall drive:
```

```
set_drive -rise 3 [all_inputs]
```

```
set_drive -fall 2 [all_inputs]
```

The drive of an input port is used to calculate the transition time at the first cell. The drive value specified is also used to compute the delay from the input port to the first cell in the presence of any RC interconnect.

```
Delay_to_first_gate =
    (drive * load_on_net) + interconnect_delay
```

The **set_driving_cell** specification offers a more convenient and accurate approach in describing the drive capability of a port. The *set_driving_cell* can be used to specify a cell driving an input port.

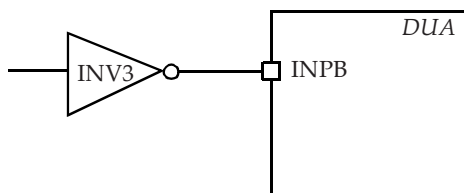


Figure 7-29 Representation for *set_driving_cell* specification example.

```
set_driving_cell -lib_cell INV3 \
    -library slow [get_ports INPB]
# The input INPB is driven by an INV3 cell
# from library slow.

set_driving_cell -lib_cell INV2 \
    -library tech13g [all_inputs]
# Specifies that the cell INV2 from a library tech13g is
# the driving cell for all inputs.

set_driving_cell -lib_cell BUFFD4 -library tech90gwc \
    [get_ports {testmode[3]}]
# The input testmode[3] is driven by a BUFFD4 cell
# from library tech90gwc.
```

Like the drive specification, the driving cell of an input port is used to calculate the transition time at the first cell and to compute the delay from the input port to the first cell in the presence of any interconnect.

One caveat of the *set_driving_cell* specification is that the incremental delay of the driving cell due to the capacitive load on the input port is included as an additional delay on the input.

As an alternate to the above approaches, the **set_input_transition** specification offers a convenient way of expressing the slew at an input port. A reference clock can optionally be specified. Here is the specification for the example shown in Figure 7-30 along with additional examples.

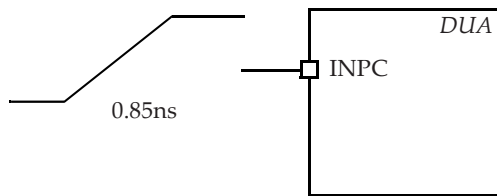


Figure 7-30 Representation for *set_input_transition* specification example.

```
set_input_transition 0.85 [get_ports INPC]
# Specifies an input transition of 850ps on port INPC.

set_input_transition 0.6 [all_inputs]
# Specifies a transition of 600ps on all input ports.

set_input_transition 0.25 [get_ports SD_DIN*]
# Specifies a transition of 250ps on all ports with
# pattern SD_DIN*.
# Min and max values can optionally be specified using
# the -min and -max options.
```

In summary, a slew value at an input is needed to determine the delay of the first cell in the input path. In the absence of this specification, an ideal transition value of 0 is assumed, which may not be realistic.

7.7.2 Modeling Capacitive Load

The **set_load** specification places a capacitive load on output ports to model the external load being driven by the output port. By default, the capacitive load on ports is 0. The load can be specified as an explicit capacitance value or as an input pin capacitance of a cell.

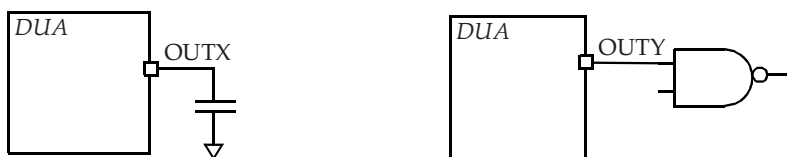


Figure 7-31 *Capacitive load on output port.*

```
set_load 5 [get_ports OUTX]
# Places a 5pF load on output port OUTX.

set_load 25 [all_outputs]
# Sets 25pF load capacitance on all outputs.

set_load -pin_load 0.007 [get_ports {shift_write[31]}]
# Place 7fF pin load on the specified output port.
# A load on the net connected to the port can be
# specified using the -wire_load option.
# If neither -pin_load nor -wire_load option is used,
# the default is the -pin_load option.
```

It is important to specify the load on outputs since this value impacts the delay of the cell driving the output. In the absence of such a specification, a load of 0 is assumed which may not be realistic.

The *set_load* specification can also be used for specifying a load on an internal net in the design. Here is an example:

```
set_load 0.25 [get_nets UCNT5/NET6]
# Sets the net capacitance to be 0.25pF.
```

7.8 Design Rule Checks

Two of the frequently used design rules for STA are *max transition* and *max capacitance*. These rules check that all ports and pins in the design meet the specified limits for transition time¹ and capacitance. These limits can be specified using:

- `set_max_transition`
- `set_max_capacitance`

As part of the STA, any violations to these design rules are reported in terms of slack. Here are some examples.

```
set_max_transition 0.6 IOBANK
# Sets a limit of 600ps on IOBANK.
```

```
set_max_capacitance 0.5 [current_design]
# Max capacitance is set to 0.5pf on all nets in
# current design.
```

The capacitance on a net is calculated by taking the sum of all the pin capacitances plus any IO load plus any interconnect capacitance on the net. Figure 7-32 shows an example.

```
Total cap on net N1 =
pin cap of UBUF1:pin/A +
```

1. As mentioned earlier, the terms “slew” and “transition time” are used interchangeably in this text.

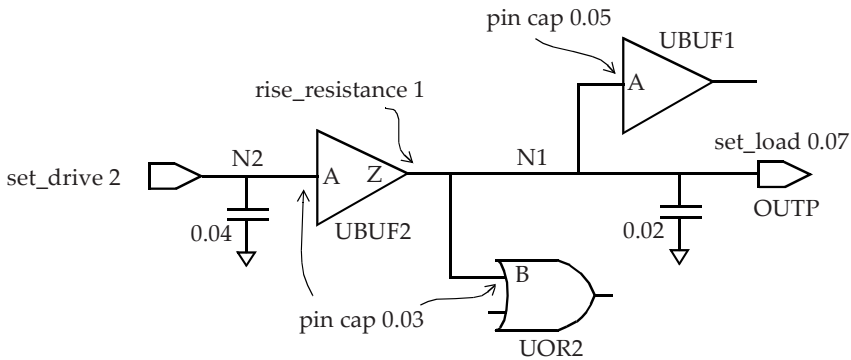


Figure 7-32 *Capacitance on various nets.*

```
pin cap of UOR2:pin/B +
load cap specified on output port OUTP +
wire/routing cap
= 0.05 + 0.03 + 0.07 + 0.02
= 0.17pF
```

Total cap on net N2 =
pin cap of UBUF2/A +
wire/routing cap from input to buffer
= 0.04 + 0.03
= 0.07pF

Transition time is computed as part of the delay calculation. For the example of Figure 7-32 (assuming linear delay model for UBUF2 cell),

Transition time on pin *UBUF2/A* =
drive of 2^1 * total cap on net *N2*
= $2 * 0.07 = 0.14\text{ns} = 140\text{ps}$

1. The library units for drive is assumed to be in Kohms.

Transition time on output port *OUTP* =
 drive resistance of *UBUF2/Z* * total cap of net *N1* =
 $1 * 0.17 = 0.17\text{ns} = 170\text{ps}$

There are other design rule checks that can also be specified for a design. These are: **set_max_fanout** (specifies a fanout limit on all pins in design), **set_max_area** (for a design); however these checks apply for synthesis and not for STA.

7.9 Virtual Clocks

A **virtual clock** is a clock that exists but is not associated with any pin or port of the design. It is used as a reference in STA analysis to specify input and output delays relative to a clock. An example where virtual clock is applicable is shown in Figure 7-33. The design under analysis gets its clock from *CLK_CORE*, but the clock driving input port *ROW_IN* is *CLK_SAD*. How does one specify the IO constraint on input port *ROW_IN* in such cases? The same issue occurs on the output port *STATE_O*.

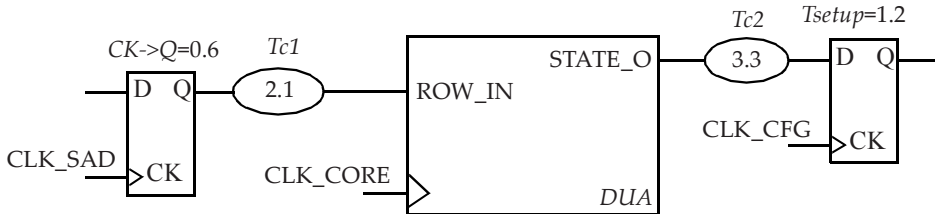


Figure 7-33 Virtual clocks for *CLK_SAD* and *CLK_CFG*.

To handle such cases, a virtual clock can be defined with no specification of the source port or pin. In the example of Figure 7-33, the virtual clock is defined for *CLK_SAD* and *CLK_CFG*.

```
create_clock -name VIRTUAL_CLK_SAD -period 10 -waveform {2 8}
```

```
create_clock -name VIRTUAL_CLK_CFG -period 8 \
-waveform {0 4}
create_clock -period 10 [get_ports CLK_CORE]
```

Having defined these virtual clocks, the IO constraints can be specified relative to this virtual clock.

```
set_input_delay -clock VIRTUAL_CLK_SAD -max 2.7 \
[get_ports ROW_IN]

set_output_delay -clock VIRTUAL_CLK_CFG -max 4.5 \
[get_ports STATE_O]
```

Figure 7-34 shows the timing relationships on the input path. This constrains the input path in the design under analysis to be 5.3ns or less.

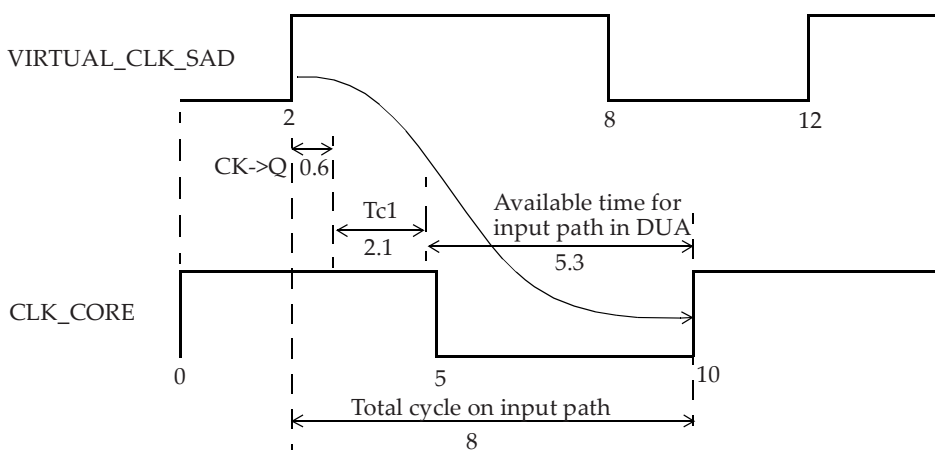


Figure 7-34 Virtual clock and core clock waveform for input path.

Figure 7-35 shows the timing relationships on the output path. This constrains the output path in the design under analysis to be 3.5ns or less.

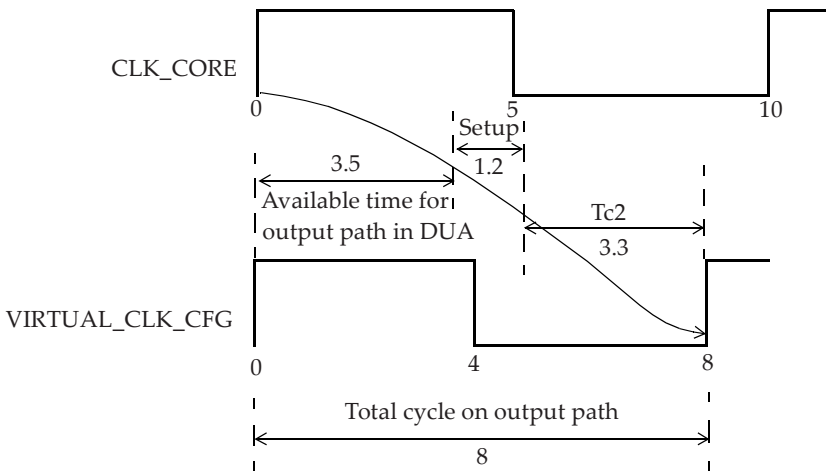


Figure 7-35 Virtual clock and core clock waveform for output path.

The *-min* option, when specified in the *set_input_delay* and *set_output_delay* constraints, is used for verifying the fast (or min) paths. The use of virtual clocks is just one approach to constrain the inputs and outputs (IO); a designer may choose other methods to constrain the IOs as well.

7.10 Refining the Timing Analysis

Four common commands that are used to constrain the analysis space are:

- i. *set_case_analysis*: Specifies constant value on a pin of a cell, or on an input port.
- ii. *set_disable_timing*: Breaks a timing arc of a cell.
- iii. *set_false_path*: Specifies paths that are not real which implies that these paths are not checked in STA.
- iv. *set_multicycle_path*: Specifies paths that can take longer than one clock cycle.

The *set_false_path* and *set_multicycle_path* specifications are discussed in greater detail in Chapter 8.

7.10.1 Specifying Inactive Signals

In a design, certain signals have a constant value in a specific mode of the chip. For example, if a chip has DFT logic in it, then the *TEST* pin of the chip should be at 0 in normal functional mode. It is often useful to specify such constant values to STA. This helps in reducing the analysis space in addition to not reporting any paths that are irrelevant. For example, if the *TEST* pin is not set as a constant, some odd long paths may exist that would never be true in functional mode. Such constant signals are specified by using the **set_case_analysis** specification.

```
set_case_analysis 0 TEST

set_case_analysis 0 [get_ports {testmode[3]}]
set_case_analysis 0 [get_ports {testmode[2]}]
set_case_analysis 0 [get_ports {testmode[1]}]
set_case_analysis 0 [get_ports {testmode[0]}]
```

If a design has many functional modes and only one functional mode is being analyzed, case analysis can be used to specify the actual mode to be analyzed.

```
set_case_analysis 1 func_mode[0]
set_case_analysis 0 func_mode[1]
set_case_analysis 1 func_mode[2]
```

Note that the case analysis can be specified on any pin in the design. Another common application of case analysis is when the design can run on multiple clocks, and the selection of the appropriate clock is controlled by multiplexers. To make STA analysis easier and reduce CPU run time, it is beneficial to do STA for each clock selection separately. Figure 7-36 shows an example of the multiplexers selecting different clocks with different settings.

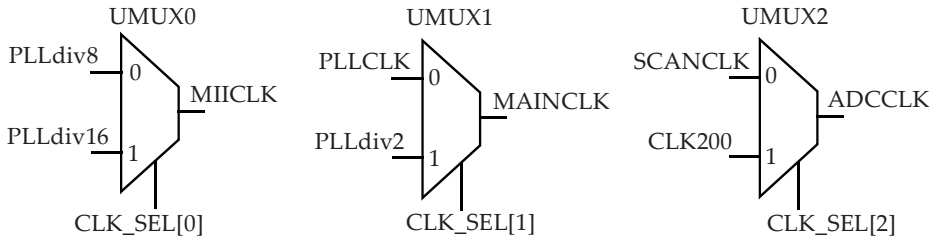


Figure 7-36 *Selecting clock mode for timing analysis.*

```
set_case_analysis 1 UCORE/UMUX0/CLK_SEL[0]
set_case_analysis 1 UCORE/UMUX1/CLK_SEL[1]
set_case_analysis 0 UCORE/UMUX2/CLK_SEL[2]
```

The first `set_case_analysis` causes `PLLdiv16` to be selected for `MIICLK`. The clock path for `PLLdiv8` is blocked and does not propagate through the multiplexer. Thus, no timing paths are analyzed using clock `PLLdiv8` (assuming that the clock does not go to any flip-flop prior to the multiplexer). Similarly, the last `set_case_analysis` causes `SCANCLK` to be selected for `ADCCLK` and the clock path for `CLK200` is blocked.

7.10.2 Breaking Timing Arcs in Cells

Every cell has timing arcs from its inputs to outputs, and a timing path may go through one of these cell arcs. In some situations, it is possible that a certain path through a cell cannot occur. For example, consider the scenario where a clock is connected to the select line of a multiplexer and the output of the multiplexer is part of a data path. In such a case, it may be useful to break the timing arc between the select pin and the output pin of the multiplexer. An example is shown in Figure 7-37. The path through the select line of multiplexer is not a valid data path. Such a timing arc can be broken by using the `set_disable_timing` SDC command.

```
set_disable_timing -from S -to Z [get_cells UMUX0]
```

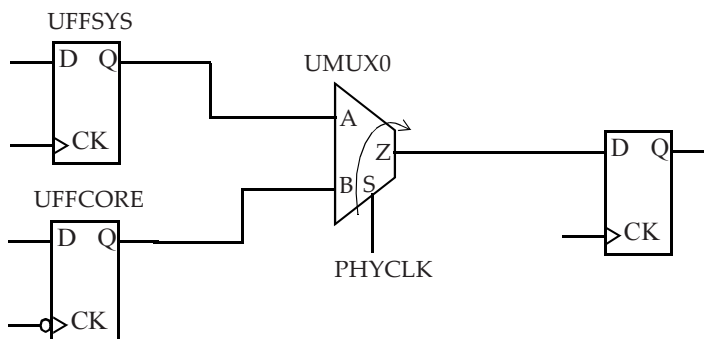


Figure 7-37 An example timing arc to be disabled.

Since the arc no longer exists, there are consequently fewer timing paths to analyze. Another example of a similar usage is to disable the minimum clock pulse width check of a flip-flop.

One should use caution when using the `set_disable_timing` command as it removes *all* timing paths through the specified pins. Where possible, it is preferable to use the `set_false_path` and the `set_case_analysis` commands.

7.11 Point-to-Point Specification

Point-to-point paths can be constrained by using the `set_min_delay` and `set_max_delay` specifications. These constrain the path delay between the from-pin and the to-pin to the values specified in the constraint. This constraint overrides any default single cycle timing paths and any multicycle path constraints for such paths. The `set_max_delay` constraint specifies the maximum delay for the specified path(s), while the `set_min_delay` constraint specifies the minimum delay for the specified path(s).

```
set_max_delay 5.0 -to UFF0/D
# All paths to D-pin of flip-flop should take 5ns max.
```

```

set_max_delay 0.6 -from UFF2/Q -to UFF3/D
# All paths between the two flip-flops should take a
# max of 600ps.
set_max_delay 0.45 -from UMUX0/Z -through UAND1/A -to UOR0/Z
# Sets max delay for the specified paths.
set_min_delay 0.15 -from {UAND0/A UXOR1/B} -to {UMUX2/SEL}

```

In the above examples, one needs to be careful that using non-standard startpoint and endpoint internal pins will force these to be the start and end points and will segment a path at those points.

One can also specify similar point-to-point constraints from one clock to another clock.

```

set_max_delay 1.2 -from [get_clocks SYS_CLK] \
  -to [get_clocks CFG_CLK]
# All paths between these two clock domains are restricted
# to a max of 1200ps.
set_min_delay 0.4 -from [get_clocks SYS_CLK] \
  -to [get_clocks CFG_CLK]
# The min delay between any path between the two
# clock domains is specified as 400ps.

```

If there are multiple timing constraints on a path, such as clock frequency, *set_max_delay* and *set_min_delay*, the most restrictive constraint is the one always checked. Multiple timing constraints can be caused by some global constraints being applied first and then some local constraints applied later.

7.12 Path Segmentation

Breaking up a timing path into smaller paths that can be timed is referred to as **path segmentation**.

A timing path has a startpoint and an endpoint. Additional startpoints and endpoints on a timing path can be created by using the *set_input_delay* and the *set_output_delay* specifications. The *set_input_delay*, which defines a startpoint, is typically specified on an output pin of a cell, while the *set_output_delay*, which defines a new endpoint, is typically specified on an input pin of a cell. These specifications define a new timing path which is a subset of the original timing path.

Consider the path shown in Figure 7-38. Once a clock is defined for *SYSCLK*, the timing path that is timed is from *UFF0/CK* to *UFF1/D*. If one is interested in reporting only the path delay from *UAND2/Z* to *UAND6/A*, then the following two commands are applicable:

```
set STARTPOINT [get_pins UAND2/Z]
set ENDPOINT [get_pins UAND6/A]
set_input_delay 0 $STARTPOINT
set_output_delay 0 $ENDPOINT
```

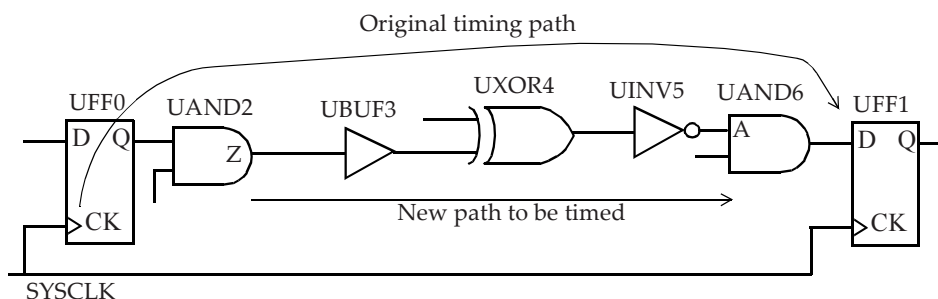


Figure 7-38 Path segmentation.

Defining these constraints causes the original timing path from *UFF0/CK* to *UFF1/D* to be segmented and creates an internal startpoint and an internal endpoint at *UAND2/Z* and *UAND6/A* respectively. A timing report would now show this new path explicitly. Note that two additional timing paths are also created automatically, one from *UFF0/CK* to *UAND2/Z* and another from *UAND6/A* to *UFF1/D*. Thus the original timing path has been broken up into three segments, each of which is timed separately.

The *set_disable_timing*, *set_max_delay* and *set_min_delay* commands also cause timing paths to get segmented.

□